

30-Minute DevOps Mini Challenge

Terraform + Ansible + Docker

Hands-on integrated lab | Maximum duration: 30 minutes

Scenario

Your team needs to deploy a simple Nginx-based web application on an AWS EC2 instance using automation only.

Goal: The application must be reachable from a browser and display:

Hello from DevOps Automation!

Tools You Must Use

- Terraform - create the AWS infrastructure
- Ansible - configure the EC2 instance
- Docker - package and run the web application

Suggested Timing

Part	Technology	Time
1	Terraform	10 min
2	Ansible	10 min
3	Docker	10 min

Task 1 - Terraform Infrastructure (10 minutes)

Create the infrastructure using Terraform only. Do not create the EC2 instance manually from the AWS Console.

- 1 EC2 instance
- 1 Security Group allowing SSH (22) and HTTP (80)
- Output the EC2 public IP
- Use variables for AWS region and instance type
- Use a data source to retrieve the latest Ubuntu AMI

Expected structure:

terraform/

├─ main.tf

├─ variables.tf

└─ outputs.tf

Task 2 - Ansible Configuration (10 minutes)

Use Ansible to configure the EC2 instance created by Terraform.

- Connect to the EC2 instance
- Update the package cache
- Install Docker
- Start and enable the Docker service
- Add the SSH user to the Docker group
- Keep the playbook idempotent - running it twice should not cause unnecessary changes

Expected structure:

```
ansible/  
├─ ansible.cfg  
├─ inventory.ini  
└─ playbook.yml
```

Run: `ansible-playbook playbook.yml`

Task 3 - Docker Application (10 minutes)

Create a small custom Nginx image and run it on the configured EC2 instance.

- Create index.html containing: Hello from DevOps Automation!
- Create a Dockerfile based on nginx:alpine
- Copy index.html to /usr/share/nginx/html/index.html
- Expose port 80
- Build the image as devops-lab:1.0
- Run the container and publish port 80
- Verify the application from a browser using the EC2 public IP

Expected structure:

```
app/  
├─ Dockerfile  
└─ index.html
```

Validation Checklist

- ☐ Terraform successfully creates the EC2 instance
- ☐ The EC2 instance accepts SSH and HTTP traffic
- ☐ Ansible installs and starts Docker successfully
- ☐ The Docker image builds without errors

- ☐ The container is running
- ☐ Opening `http://<EC2-PUBLIC-IP>` displays the expected message

Bonus Challenge - Finish Early?

Modify the Ansible playbook so that Ansible also builds and runs the Docker container automatically.

Terraform → EC2 → Ansible → Docker → Running Application → Browser